

Incremental Structure-from-Motion with Adaptive Optimization: A Virtual Tour Implementation

Muhammad Hussain Habib
Student ID: 27100016
hussain.habib@example.edu

Ayaan Ahmed
Student ID: 27100155
ayaan.ahmed@example.edu

December 7, 2025

Abstract

We present a complete Structure-from-Motion (SfM) pipeline for 3D scene reconstruction and interactive virtual tour generation. Our system processes sequential images to recover camera poses and sparse 3D geometry through incremental registration using Perspective-n-Point (PnP) estimation, followed by iterative Bundle Adjustment for global consistency. The pipeline addresses key challenges including $O(N^2)$ matching complexity through temporal locality constraints, tracking failures via adaptive PnP thresholds, and geometric instability through angle-aware triangulation filtering. We validate our approach on a 332-frame dataset, producing a point cloud exceeding 200,000 3D points. The reconstruction is integrated into an interactive WebGL-based virtual tour using Three.js, enabling smooth navigation between camera viewpoints through interpolated transitions. Full implementation and demonstration video available at: https://github.com/Atlantis004/Computer_Vision_Project

1 Introduction

Structure-from-Motion (SfM) reconstructs three-dimensional scene geometry and camera trajectories from unordered image collections, forming the foundation of modern photogrammetry applications ranging from cultural heritage preservation to autonomous navigation. While commercial solutions like Matterport and Reality Capture provide robust reconstruction capabilities, understanding the underlying geometric principles and implementation challenges remains crucial for computer vision practitioners.

This work implements a complete incremental SfM pipeline that transforms sequential video frames into navigable 3D environments. Unlike batch reconstruction methods that process all images simultaneously, our incremental approach mirrors online SLAM systems by registering cameras sequentially through 2D-3D correspondences. This paradigm enables real-time operation but introduces unique challenges in drift accumulation, loop closure detection, and computational efficiency.

ciency.

Key Contributions:

- An adaptive incremental SfM system with temporal locality constraints that reduces matching complexity from $O(N^2)$ to approximately $O(N)$
- Angle-aware triangulation filtering and survival-mode PnP for robust tracking through difficult sequences
- Sparse Bundle Adjustment formulation with local windowing for efficient global optimization
- Complete pipeline integration into an interactive Three.js virtual tour with smooth camera interpolation

The remainder of this paper is organized as follows: Section 2 details the mathematical formulation of each pipeline stage, Section 3 presents experimental validation, Section 4 analyzes key implementation challenges and solutions, Section 5 describes the virtual tour architecture, and Section 6 concludes with lessons learned.

2 Methodology

2.1 Feature Extraction and Matching

The pipeline begins by detecting Scale-Invariant Feature Transform (SIFT) keypoints in each frame. SIFT provides robustness to scale, rotation, and illumination changes through its multi-scale representation and gradient-based descriptors. For each image I_i , we extract up to 40,000 keypoints $\{\mathbf{p}_j\}$ with corresponding 128-dimensional descriptors $\{\mathbf{d}_j\}$.

Feature matching between frames I_i and I_k employs a brute-force matcher with $k = 2$ nearest neighbors, followed by Lowe’s ratio test to reject ambiguous matches:

$$\frac{\|\mathbf{d}_i - \mathbf{d}_{k,1}\|}{\|\mathbf{d}_i - \mathbf{d}_{k,2}\|} < \tau_{\text{ratio}} \quad (1)$$

where $\mathbf{d}_{k,1}$ and $\mathbf{d}_{k,2}$ are the first and second nearest neighbors, and $\tau_{\text{ratio}} = 0.75$. This threshold filters approximately 70% of initial correspondences, retaining only distinctive matches.

To address the $O(N^2)$ complexity of all-pairs matching in sequential data, we implement a sliding window approach with temporal locality. Each new frame I_t is matched only against frames within a search radius $[t - r, t]$ where $r = 20$, and matching terminates early if the match count exceeds 600 inliers. For video sequences, this exploits the temporal coherence assumption that consecutive frames share substantial overlap.

2.2 Camera Intrinsic Calibration

Accurate camera intrinsics are critical for metric reconstruction. Rather than assuming generic focal lengths, we extract the intrinsic matrix $\mathbf{K}$ from image EXIF metadata when available, or perform checkerboard calibration:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (2)$$

For high-resolution (4K) imagery, we additionally estimate radial distortion coefficients (k_1, k_2) and apply `cv2.undistort` preprocessing to eliminate the characteristic “bowl effect” where planar surfaces appear curved. This correction proved essential for geometric consistency across long sequences.

2.3 Two-View Initialization

Map initialization selects two frames with sufficient baseline to establish metric scale. Given matched normalized coordinates $\{\hat{\mathbf{p}}_i\}$ and $\{\hat{\mathbf{p}}'_i\}$ where $\hat{\mathbf{p}} = \mathbf{K}^{-1}\mathbf{p}$, we estimate the Essential matrix $\mathbf{E}$ using RANSAC:

$$\hat{\mathbf{p}}'^T \mathbf{E} \hat{\mathbf{p}} = 0 \quad (3)$$

The Essential matrix encodes the relative geometry between views and decomposes into rotation $\mathbf{R}$ and translation $\mathbf{t}$ (up to scale):

$$\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R} \quad (4)$$

where $[\mathbf{t}]_{\times}$ is the skew-symmetric matrix of $\mathbf{t}$.

Decomposing $\mathbf{E}$ yields four candidate solutions. We perform cheirality checking by triangulating points under each hypothesis and selecting the pose $[\mathbf{R}|\mathbf{t}]$ that places the maximum number of points in front of both cameras (positive depth $Z > 0$).

Initial triangulation uses the Direct Linear Transform (DLT) on projection matrices $\mathbf{P}_1 = \mathbf{K}[\mathbf{I}|0]$ and $\mathbf{P}_2 = \mathbf{K}[\mathbf{R}|\mathbf{t}]$ to recover homogeneous world coordinates $\mathbf{X} = [X, Y, Z, W]^T$, which are normalized to Euclidean form $\mathbf{X} = [X/W, Y/W, Z/W]^T$.

2.4 Incremental Camera Registration

Once initialized, subsequent frames are registered through Perspective-n-Point (PnP) estimation. For each candidate frame, we establish 2D-3D correspondences by matching its keypoints against features from the most recently added camera that are already associated with 3D map points.

Given n correspondences $\{(\mathbf{p}_i, \mathbf{X}_i)\}$ where $\mathbf{p}_i$ are image observations and $\mathbf{X}_i$ are world points, PnP solves for the camera pose $[\mathbf{R}|\mathbf{t}]$ by minimizing reprojection error:

$$\min_{\mathbf{R}, \mathbf{t}} \sum_{i=1}^n \|\mathbf{p}_i - \pi(\mathbf{K}[\mathbf{R}|\mathbf{t}]\mathbf{X}_i)\|^2 \quad (5)$$

where π denotes perspective projection to image coordinates.

We employ `cv2.solvePnP` with adaptive thresholds. During nominal operation, we require $\epsilon_{\text{reproj}} = 8.0$ pixels and a minimum of 15 inliers. However, when tracking degrades (fast rotations, motion blur), the system enters “survival mode” by relaxing constraints to $\epsilon_{\text{reproj}} = 20.0$ pixels and 6 inliers, allowing the tracker to maintain continuity through difficult segments.

After successful registration, we triangulate new 3D points by matching the current frame against previous cameras. To avoid geometric instability from near-parallel rays, we implement angle-aware filtering:

$$\theta_{\text{tri}} = \arccos\left(\frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\| \|\mathbf{v}_2\|}\right) > \theta_{\min} \quad (6)$$

where $\mathbf{v}_1$ and $\mathbf{v}_2$ are viewing rays from two cameras to the triangulated point, and $\theta_{\min} = 2.0$. Points failing this criterion are rejected to prevent depth explosion artifacts.

2.5 Bundle Adjustment

Incremental reconstruction accumulates drift as small errors compound across sequential registrations. Bundle Adjustment (BA) performs joint optimization of all camera parameters and 3D point positions by minimizing global reprojection error:

$$\min_{\{\mathbf{R}_j, \mathbf{t}_j\}, \{\mathbf{X}_k\}} \sum_{j,k} \rho(\|\mathbf{p}_{jk} - \pi(\mathbf{K}[\mathbf{R}_j|\mathbf{t}_j]\mathbf{X}_k)\|^2) \quad (7)$$

where ρ is the Huber loss for robustness to outliers, and the sum runs over all observations $\mathbf{p}_{jk}$ of point k in camera j .

Direct application of dense nonlinear least squares to problems with 100+ cameras and 60,000+ points causes memory exhaustion. We exploit the sparse structure inherent in SfM: each observation depends only on one camera and one point, yielding a Jacobian with $< 1\%$ non-zero entries.

We parameterize rotations using Rodrigues representation $\boldsymbol{\omega}$ and construct the parameter vector $\boldsymbol{\theta} = [\boldsymbol{\omega}_1, \mathbf{t}_1, \dots, \mathbf{X}_1, \dots]^T$. The Jacobian sparsity pattern is encoded as a binary matrix indicating which residuals depend on which parameters, enabling

`scipy.optimize.least_squares` to use sparse linear algebra.

Additionally, we implement local BA by optimizing only an active window of the 15 most recent cameras while holding older cameras fixed. This reduces dimensionality and focuses refinement on the evolving reconstruction frontier. BA is triggered every 5 new frames and once globally at completion.

3 Experiments and Results

3.1 Dataset and Implementation

We collected a 332-frame video sequence of a building interior using a smartphone camera at 1920×1080 resolution. The trajectory included corridor traversal with multiple 90° turns and varying lighting conditions. Camera intrinsics were extracted from EXIF metadata, yielding $f_x = f_y \approx 1200$ pixels and principal point at image center.

The pipeline was implemented in Python using OpenCV 4.8, NumPy, SciPy, and Open3D for visualization. Processing utilized an Intel i7 CPU with 16GB RAM. Feature extraction took approximately 0.8 seconds per frame, while PnP registration averaged 0.3 seconds. Local BA windows required 15-30 seconds depending on the number of points.

3.2 Validation Results

Initial validation used a 30-frame subset to verify pipeline correctness before scaling to the full dataset. Two-view initialization between frames 0 and 15 (wide baseline) produced 2,527 initial points. Incremental reconstruction successfully registered 27 of 30 frames, with 3 frames rejected due to insufficient overlap during a rapid turn.

The resulting point cloud contained 136,000 3D points spanning approximately 20×15 meters. Visual inspection revealed clean reconstruction of walls, doorways, and architectural features. Quantitative evaluation showed mean reprojection error decreasing from 12.3 pixels before BA to 2.1 pixels after global optimization.

For production, we processed the full 332-frame sequence using Agisoft Metashape, yielding a dense point cloud exceeding 200,000 points with accurate camera poses. This dataset was exported for the virtual tour implementation.

3.3 Computational Efficiency

Table 1 summarizes timing improvements from our optimizations:

The sliding window matcher reduced per-frame matching from over 20 minutes (exhaustive search across 100 frames) to approximately 15 seconds. Sparse BA with local windowing decreased optimization time from 90+ minutes for global problems to 30 seconds for active windows of 15 cameras.

Table 1: Computational performance improvements

Operation	Naive	Optimized
Feature matching	20+ min	15 sec
Bundle Adjustment	90+ min	30 sec

4 Discussion

4.1 Combinatorial Matching Complexity

The most significant computational bottleneck emerged from naive all-pairs feature matching. With 20,000 features per frame and 100 frames, brute-force matching attempts $100 \times 20000 \times 20000 \approx 4 \times 10^{10}$ descriptor comparisons. For sequential video, this is wasteful: frame 0 shares negligible overlap with frame 100.

Our solution exploits temporal coherence through a sliding window ($r = 20$ frames) and early termination (600+ matches). This reduces complexity from $O(N^2)$ to approximately $O(Nr)$ where $r \ll N$, achieving 80× speedup. The greedy exit strategy recognizes that “good enough” matches suffice for robust registration, avoiding exhaustive search for the globally optimal correspondence set.

4.2 Tracking Failure and Recovery

Fast camera rotations (e.g., 90° turns at building corners) caused temporary tracking loss due to motion blur and limited overlap. The matcher would find zero correspondences with recent frames, then incorrectly match against a visually similar but spatially distant frame hundreds of frames away, causing “teleportation” artifacts in the trajectory.

We addressed this through two mechanisms. First, survival-mode PnP relaxes inlier thresholds ($\epsilon = 20$ px, 6 inliers) to maintain tracking through difficult segments, accepting temporarily degraded geometry that BA will later refine. Second, we enforce frontier search by restricting candidate frames to the leading edge of the reconstruction ($t_{\max} - r \leq t \leq t_{\max}$), preventing false loop closures during continuous forward motion.

4.3 Geometric Instability

Initial reconstructions exhibited severe artifacts: a shapeless “mound of dirt” point cloud, planar floors curving into bowls, and straight walls appearing bent. These issues stemmed from three sources.

Small baseline triangulation: Consecutive video frames provide minimal parallax (camera translation ≈ 5 cm between frames). With near-parallel viewing rays, small pixel noise amplifies into massive depth errors. For example, rays differing by 0.5° intersecting 10 meters away yield 9 cm uncertainty, but at 100 meters this becomes 87 cm.

We mitigate this through angle-aware filtering ($\theta > 2.0$), rejecting points from near-degenerate geometry. Additionally, we force wide-baseline initialization (frames 0 and 15) to establish robust initial scale. However, strict angle thresholds can starve the map of new points, so we implement an adaptive “survival pass” at $\theta > 0.5$ when accumulation stalls.

Uncalibrated distortion: Lens radial distortion causes straight lines to curve, manifesting as the bowl effect. Initially assuming pinhole cameras, our reconstruction inherited these distortions. Checkerboard calibration to recover distortion coefficients (k_1, k_2) and preprocessing with `cv2.undistort` eliminated this artifact entirely.

Resolution scaling: High-resolution (4K) images failed PnP despite 1000+ matches. Investigation revealed that a fixed 8-pixel reprojection threshold is reasonable at 1200px width but overly strict at 4000px (0.2% vs 0.7% of image dimension). Scaling thresholds proportionally to resolution ($\epsilon = 8 \times w/1200$) restored registration success.

4.4 Bundle Adjustment Scalability

Global BA on 100+ cameras and 60,000+ points constructed dense Jacobian matrices exceeding available RAM (16GB), causing kernel crashes. The fundamental issue is that `scipy`’s default dense solver allocates $O(M^2)$ memory for M parameters, despite $< 1\%$ of entries being non-zero.

Our sparse formulation explicitly encodes the structure that observation i depends only on camera $c(i)$ and point $p(i)$, producing a Jacobian with banded structure. Representing this as a sparse matrix (`scipy.sparse.lil_matrix`) reduces memory from gigabytes to megabytes, enabling optimization to complete.

Local BA further improves efficiency by optimizing only an active window of recent cameras. The intuition is that cameras 0-10 have stabilized by the time the reconstruction reaches frame 100, so re-optimizing them provides diminishing returns. Fixing older cameras as anchors and optimizing the frontier window of 15 frames maintains accuracy while reducing problem dimensionality 6-fold.

We also impose iteration limits (`max_nfev=20`) and relaxed tolerance (`ftol=1e-2`) to trade marginal accuracy gains for practical runtime. This philosophy recognizes that “perfect” optimization taking hours provides no advantage over “good enough” optimization completing in seconds for interactive applications.

5 Virtual Tour Implementation

5.1 Coordinate System Transformation

OpenCV and Three.js use different coordinate conventions, requiring careful transformation of camera poses. OpenCV adopts a right-handed system with Z forward, X right, and Y

down, while Three.js (following OpenGL) uses Z backward, X right, and Y up.

Camera poses exported from SfM are converted by constructing 4×4 transformation matrices that apply a 180° rotation about the X axis:

$$\mathbf{T}_{\text{three}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{T}_{\text{cv}} \quad (8)$$

This ensures consistent orientation between the reconstructed scene and the rendered environment.

5.2 Waypoint Visualization

Each camera position is represented by a directional cone mesh oriented along the camera’s viewing direction. We construct these by:

- Creating `ConeGeometry(0.3, 0.8, 16)` and rotating $-\pi/2$ radians about X to point forward
- Applying the corrected camera quaternion to align with viewing direction
- Positioning at the camera location offset downward by 1.5 units to appear grounded

Cones use semi-transparent yellow material (`opacity: 0.6`) with `depthTest: false` to remain visible through occlusions. On hover, cones scale to $1.3\times$ and change to red, providing clear interaction feedback.

5.3 Navigation Mechanics

The virtual tour supports three navigation modes: walking (WASD/arrow keys), teleportation (double-click), and overview (H key toggle).

Walking mode: Keyboard input is processed each frame by computing forward and right direction vectors projected to the horizontal plane. Movement accumulates as:

$$\mathbf{p}_{\text{new}} = \mathbf{p} + v\Delta t(\mathbf{d}_f w_f + \mathbf{d}_r w_r) \quad (9)$$

where $v = 0.15$ is walk speed, $\mathbf{d}_f$ and $\mathbf{d}_r$ are unit forward and right vectors, and $w_f, w_r \in \{-1, 0, 1\}$ encode key states.

Teleportation: Raycasting detects intersections with either the point cloud or an invisible ground plane. Successful hits trigger animated camera translation using `TWEEN.js` with quadratic easing over 1 second. The orbit control target moves with the camera to maintain view direction.

Photo mode: Single-clicking a waypoint cone triggers:

1. Smooth camera animation to the exact waypoint pose (position + orientation)

2. Linear interpolation (lerp) of position: $\mathbf{p}(t) = (1-t)\mathbf{p}_0 + t\mathbf{p}_1$
3. Spherical linear interpolation (slerp) of rotation: $\mathbf{q}(t) = \text{slerp}(\mathbf{q}_0, \mathbf{q}_1, t)$
4. Cross-fade of overlay image from opacity 0 to 0.9 over 1.5 seconds
- [4] V. Lepetit et al., “EPnP: An accurate $O(n)$ solution to the PnP problem,” *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009.
- [5] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [6] R. Cabello et al., “Three.js,” <https://threejs.org>, 2010-2024.

Slerp ensures constant angular velocity during rotation transitions, avoiding gimbal lock and providing visually smooth camera reorientation. The overlay image uses `pointer-events: none` to allow interaction with the 3D scene beneath.

5.4 Frame-Specific Rotation Corrections

Video frames 273-331 exhibited inconsistent orientation due to camera rotation during recording. Rather than reprocessing the entire SfM pipeline, we apply runtime corrections in the viewer by detecting frame numbers via regex on filenames and conditionally applying an additional $-\pi/2 - 0.5$ radian rotation about the Y axis to these waypoints.

6 Conclusion

This project demonstrates a complete SfM pipeline from image capture through interactive visualization. Key lessons include the critical importance of temporal locality for computational efficiency, the necessity of multi-stage validation (two-view, incremental, global BA), and the value of adaptive thresholds for robust tracking.

The evolution from naive global matching to an adaptive sliding-window system with survival logic, sparse optimization, and local refinement represents the practical engineering required to transition theoretical algorithms into functional systems. Our final reconstruction of 200,000+ points from 332 frames validates the approach, while the Three.js virtual tour demonstrates practical applications of SfM beyond static point cloud visualization.

Future work includes dense reconstruction via multi-view stereo, automatic loop closure detection for larger environments, and real-time operation through GPU acceleration and incremental BA solvers.

References

- [1] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [2] J. L. Schonberger and J.-M. Frahm, “Structure-from-motion revisited,” in *CVPR*, 2016.
- [3] B. Triggs et al., “Bundle adjustment — a modern synthesis,” in *Vision Algorithms: Theory and Practice*, pp. 298–372, 2000.